

Developer handover

The design and the code are done. Your job is to put them on a staging copy of the live store, prove them against real WooCommerce, load the real content, and run the launch. Nothing here asks you to build new features.

From Ateeshay Jain, Mindlayer

23 September 2026

Repository: github.com/ateeshayjain/Letsfoodify · branch `main`

Five rules that cause irreversible damage if broken

1. **The live site holds the order history.** Never run anything against production without showing Ateeshay the exact command and getting a yes.
2. **Never copy the staging database to production.** Live will have weeks of orders staging never saw. Settings move through `build/scripts/bootstrap.sh` — if you change something in wp-admin, add it to that script the same day, or it is lost at launch.
3. **The smoke test is blocking.** If `smoke-test.sh` exits non-zero, nothing is deployed. No overrides.
4. **No hard-coded colour, font size or spacing.** Every value comes from `theme.json`; add a token there first. `tests/token-test.py` checks that every token used actually exists.
5. **Ask before installing any paid plugin.** The client's yearly budget is already at its limit.

Before and after every change, run the gate: `build/run-all-tests.sh` — 30 suites. A suite that cannot run (no PHP, no browser, no WordPress) counts as a failure, not a skip, so green means verified. Full background and the reasoning behind each rule: `CLAUDE.md` at the repo root.

The five steps

1 Access, and protect the live store DAY ONE

- ☐ Hosting access (Hostinger hPanel), a WordPress admin login, and SSH with WP-CLI.
- ☐ Take a fresh backup of the live site — **and restore it on a scratch site to prove it works.** A backup that has never been restored is a hope, not a backup.
- ☐ Confirm with Ateeshay that the client has started the slow approvals: DLT registration for SMS login, Google Business Profile verification, the photo shoot. They take weeks and run in the background.

2 Install on a staging copy ABOUT AN AFTERNOON

Follow `build/docs/INSTALL-ON-STAGING.md` line by line. In short:

- ☐ Create a staging copy of the live store (hPanel → WordPress → Staging) and tick *Discourage search engines*.
- ☐ Upload the theme zip — Appearance → Themes → Add New → Upload. Use the zip sent with this document, or build one with `build/scripts/package-theme.sh`, which boots the zip in WordPress before calling it good.
- ☐ From the staging WordPress folder, run `bootstrap.sh --env=staging --dry-run`, read it, then run it for real.
- ☐ Run the smoke test against staging with `--staging`. It must pass.

3 Prove it against real WooCommerce AT LEAST A FULL DAY

This is the part nobody has done yet. The theme has been booted in real WordPress, but WooCommerce could not be installed where it was built, so the shop is first seen working here.

- ☐ **Plugin names and Rank Math settings.** Written from memory — check each one resolves in the dry run. A wrong settings key fails silently: the setting simply never applies.
- ☐ **A real test order** through Razorpay in test mode, plus one cash-on-delivery order, then cancel it. Check the nine-field checkout: removing “last name” may break Razorpay or the GST invoice. If it does, keep the field hidden and fill it from the full name.
- ☐ **Every shop screen** against `build/docs/Foodify-Storefront-Design.pdf`: product grids, filters and sort, the header cart, cart, checkout, account, a combo box.
- ☐ **Real phones** — iPhone Safari and Android Chrome. The automated sweep covers layout; fonts, the menu overlay and the payment popup only show on real devices.
- ☐ The analytics check in `docs/WP-13-NOTES.md`: view → add to cart → checkout → purchase, and purchase fires exactly once on refresh.

4 Load the real content AS THE CLIENT SUPPLIES IT

- ☐ Photos — four per product, in order: pack front, prepared in a bowl, ingredients laid out, in a hand or beside a mug.
- ☐ Each product's fields in the product editor: ingredients, allergens, net quantity, cooked weight, taste notes, FAQ, HSN code and GST rate. An empty required declaration shows as “Not provided” on the page — on purpose.
- ☐ Combos: *Combo* — *what's inside* (one meal per line) and *Combo* — *cost bought separately*. No separate price means no saving is shown.
- ☐ FSSAI licence number and phone number in `theme/foodify/inc/business-profile.php`. Until then the site prints **NOT CONFIGURED** — correct, and never to be replaced with a placeholder.
- ☐ Tag clean-up — `tags-to-attributes.php`, then `taxonomy-cleanup.php`, each via `wp eval-file ... report` first. Deletions happen only with `execute --confirm`.

5 Launch ONE AGREED WINDOW

Follow `build/docs/LAUNCH-RUNBOOK.md` — it is the whole sequence.

- ☐ A full dress rehearsal on staging the day before, run as if it were the launch.
- ☐ At launch only **files and `bootstrap.sh`** move to the live site. Never the staging database.
- ☐ Smoke test before and after. Keep the old host live and paid for 30 days — it is the rollback.

Known risks — check these first

From `build/docs/REVIEW-NOTES.md`, where each is written up in full.

WooCommerce never run **HIGH**

Every WooCommerce block, the cart, checkout and payment paths are unverified until staging. Budget the full day in step 3.

Last-name field **HIGH**

Razorpay and some invoice plugins expect a surname. Test a real transaction before trusting the nine-field checkout.

Slugs and option keys from memory **MEDIUM**

`seo-by-rank-math`, `judgeme-product-reviews-woocommerce` and the Rank Math title keys in `bootstrap.sh`. Confirm each in the dry run.

PIN-code lookup is third-party **MEDIUM**

Checkout calls `api.postalpincode.in` from the customer's browser. It fails quietly, but the client must agree and the privacy policy needs a line. An offline PIN dataset avoids both.

“Only N left” needs stock **MEDIUM**

The low-stock line only appears if stock is managed per product. Check whether the client does.

SMS login waits on DLT **MEDIUM**

Mobile-number sign-in needs DLT registration. Until then, email and password sign-in stands in; guest checkout is the default.

What you will get from the client

ITEM	NEEDED FOR	WHO
FSSAI licence number	Header strip, every product page, invoices	Client
Phone number	Footer and local business data — must match Google Business Profile exactly	Client
Product photography	Every product page and the Google product listing	Client — shoot booked
Per-product details	Ingredients, allergens, weights, taste, FAQ, HSN, GST rate	Client, HSN/GST from their CA
Combo boxes	Which boxes exist, contents, price and cost bought separately	Client
Razorpay keys	Test keys for staging, live keys at launch	Client
DLT registration and SMS provider	Mobile-number sign-in	Client — takes weeks
Google Business Profile	Reviews on the home page, local search	Client

Not yours to decide — wait for these

- ☐ **Host move.** Re-test the live site's caching in a clean browser first; the original reading was likely skewed by a cart cookie. If it says HIT, the move is optional.
- ☐ **Paid plugins** (Rank Math Pro, an OTP plugin, image optimisation): the yearly total goes to the client before any is installed.
- ☐ **Timeline.** The one-developer plan is in `build/docs/SOLO-PLAN.md`. Agree dates with Ateeshay before quoting any.

Commands

```
# the gate – before and after every change (add a URL to include the blocking smoke test)
build/run-all-tests.sh
build/run-all-tests.sh https://<staging-url>

# the theme zip – refuses to produce one that does not boot in WordPress
build/scripts/package-theme.sh

# configuration on staging – run FROM the staging WordPress folder
bash ~/letsfoodify/build/scripts/bootstrap.sh --env=staging --dry-run
bash ~/letsfoodify/build/scripts/bootstrap.sh --env=staging

# the blocking gate against staging
bash ~/letsfoodify/build/scripts/smoke-test.sh https://<staging-url> --staging

# the preview and the client PDF, after a design change
python3 build/tools/render-preview.py
node build/tools/design-book/capture.js && python3 build/tools/design-book/build.py && node
build/tools/design-book/render.js
```

Where things live

PATH	WHAT IT IS
CLAUDE.md	Start here: the rules, the design tokens, and why each call went the way it did
build/theme/foodify/	The theme — a standalone block theme, no page builder, no parent theme
build/scripts/bootstrap.sh	Every setting, as code. The only way configuration reaches the live site
build/scripts/smoke-test.sh	The blocking launch gate
build/docs/INSTALL-ON-STAGING.md	Step 2, in full
build/docs/LAUNCH-RUNBOOK.md	Step 5: cutover, rollback, the 30 days after
build/docs/REVIEW-NOTES.md	The known risks above, written up in full
build/docs/WP-*-NOTES.md	One file per work package: what was built and why
build/docs/Foodify-Storefront-Design.pdf	What the finished site should look like, screen by screen

If something goes wrong

On staging: Appearance → Themes → activate the old theme. That is the whole rollback — products and orders were never touched, and `bootstrap.sh` exported the database to `../backups/` before it changed anything. **On the live site:** the rollback section of `LAUNCH-RUNBOOK.md`, and nothing else.

Questions go to Ateeshay Jain at Mindlayer before anything touches the live site — rule 1 is not a formality.